

HormuzNet: Monitoramento Marítimo Descentralizado e Resiliência em Sistemas Críticos

Luis Felipe Pereira de Carvalho¹

¹Departamento de Tecnologia – Universidade Estadual de Feira de Santana (UEFS)
44036-900 – Feira de Santana – Bahia

luisoftbr@outlook.com.br

Resumo. *Este trabalho apresenta o HormuzNet, um sistema de monitoramento marítimo descentralizado desenvolvido para garantir a resiliência e a disponibilidade de dados no Estreito de Ormuz. A solução substitui arquiteturas centralizadas por uma malha autônoma de brokers que utilizam o Relógio Lógico de Lamport para ordenação global de eventos e exclusão mútua distribuída. A comunicação é baseada em protocolos nativos: UDP Multicast para aquisição redundante de dados de sensores, TCP persistente com lógica de fallback para coordenação de drones autônomos e WebSocket para visualização tática em tempo real. O sistema inclui um modelo de falhas probabilístico para drones e foi validado em ambiente containerizado, demonstrando capacidade de recuperação automática diante da queda de nós e consistência no despacho de missões.*

1. Introdução

A região do Estreito de Ormuz é um dos pontos mais críticos do comércio global, exigindo monitoramento constante e capacidade de resposta rápida a incidentes marítimos. Tradicionalmente, sistemas de monitoramento operam sob modelos centralizados, onde uma falha no servidor central pode paralisar toda a operação de vigilância e resgate. O projeto HormuzNet surge para endereçar essa vulnerabilidade através de uma arquitetura de sistemas distribuídos descentralizada.

O objetivo principal deste trabalho é descrever a implementação de uma malha autônoma e resiliente, onde múltiplos *brokers* cooperam para processar dados de sensores ambientais e coordenar frotas de drones autônomos. A solução foca na eliminação de pontos únicos de falha e na garantia de consistência eventual através de algoritmos clássicos de sistemas distribuídos, como os Relógios de Lamport.

O sistema é composto por três tipos de entidades: sensores, que publicam dados via UDP Multicast; *brokers*, que formam a malha de controle P2P; e drones autônomos, que executam missões de resposta. Este documento detalha as decisões de projeto, a implementação da concorrência em linguagem Go, os protocolos de rede utilizados e os resultados obtidos na validação da resiliência do sistema.

2. Fundamentação Teórica

2.1. Sistemas Distribuídos e Descentralização

Sistemas centralizados apresentam riscos elevados em ambientes críticos devido à existência de pontos únicos de falha (*Single Points of Failure* - SPOF)

[Tanenbaum et al. 2021]. A descentralização distribui a autoridade e o processamento entre múltiplos nós, aumentando a disponibilidade e a resiliência. No HormuzNet, a remoção de bases estáticas e a adoção de uma malha (*mesh*) de *brokers* permite que o sistema continue operacional mesmo que parte da infraestrutura seja comprometida.

2.2. Ordenação de Eventos e Relógio de Lamport

Em um sistema descentralizado, não existe um relógio físico global confiável para ordenar eventos ocorridos em diferentes máquinas. Leslie Lamport propôs o conceito de Relógios Lógicos [Lamport 1978] para estabelecer uma relação de "aconteceu antes" (*happened-before*). Cada nó mantém um contador local que é incrementado a cada evento e sincronizado através de mensagens. No HormuzNet, o Relógio de Lamport é utilizado para garantir que todos os *brokers* processem as ocorrências de sensores na mesma ordem lógica, permitindo uma tomada de decisão consistente sobre qual drone deve ser despachado.

2.3. Comunicação em Grupo com UDP Multicast

O protocolo UDP Multicast permite o envio de datagramas para um grupo de destinatários interessados de forma eficiente [Deering 1989]. Diferente do *Unicast*, onde o emissor precisa enviar uma cópia para cada receptor, o *Multicast* utiliza a infraestrutura de rede para replicar o pacote apenas quando necessário. Esta característica é ideal para o HormuzNet, pois permite que sensores publiquem dados uma única vez e todos os *brokers* da malha recebam a informação simultaneamente, garantindo redundância imediata sem sobrecarregar os sensores.

2.4. Resiliência e Fallback em Conexões TCP

O Transmission Control Protocol (TCP) fornece um canal bidirecional confiável e orientado à conexão [Institute 1981]. Para a coordenação de drones, a confiabilidade é essencial para garantir a entrega de ordens de missão. Entretanto, a persistência da conexão TCP em sistemas dinâmicos exige mecanismos de *fallback*. No projeto, drones implementam uma lógica de reconexão automática: ao perderem o contato com seu *broker* primário, tentam se associar a outros nós da malha, garantindo que a frota permaneça sob comando enquanto houver ao menos um *broker* ativo.

2.5. Concorrência e Paralelismo em Go

A linguagem Go foi projetada para sistemas concorrentes, utilizando o modelo de *Communicating Sequential Processes* (CSP) [Donovan and Kernighan 2015]. Através de *goroutines* (fios de execução leves) e *channels* (canais de comunicação), é possível gerenciar milhares de conexões e eventos simultâneos com baixo custo computacional. O HormuzNet explora esses recursos para processar datagramas UDP, conexões TCP de drones e sessões WebSocket de monitoramento de forma independente e segura, utilizando primitivas de sincronização (*mutexes*) para proteger o estado global compartilhado entre as *goroutines*.

3. Metodologia, Implementação e Testes

3.1. Arquitetura da Malha Descentralizada

A arquitetura do HormuzNet foi projetada como uma malha (*mesh*) autônoma de três camadas, conforme ilustra a Figura 1. A camada de aquisição utiliza sensores que publicam alertas em um endereço de *UDP Multicast*. A camada de controle é composta por

nove *brokers* distribuídos geograficamente (B1 a B9 correspondendo a Noroeste, Norte, Nordeste, Leste, Sudeste, Sul, Sudoeste, Oeste e Centro/Líder de Descoberta), que formam uma rede P2P estável em malha cooperativa. A camada de atuação consiste em sete drones autônomos que gerenciam conexões TCP persistentes com a malha.

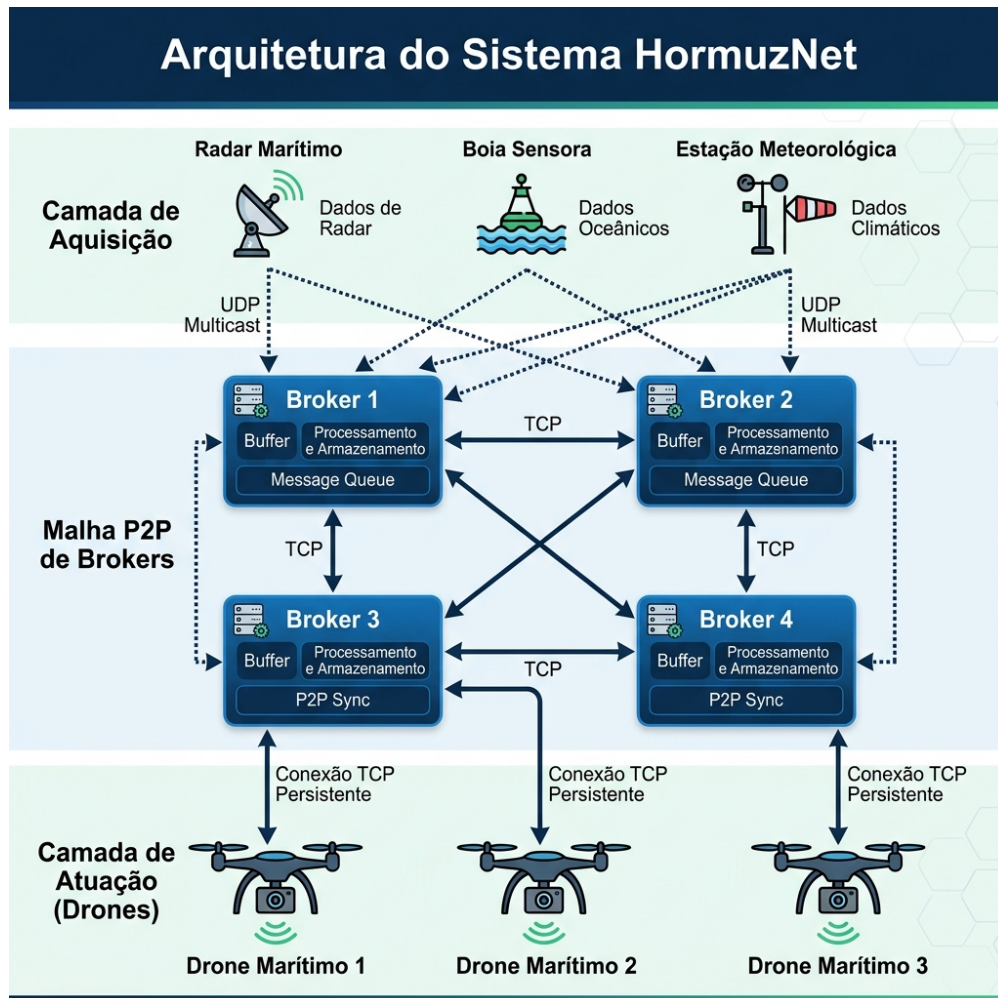


Figure 1. Arquitetura descentralizada do HormuzNet: malha P2P entre brokers e descoberta de novos nós. Fonte: próprio autor.

3.2. Componentes e Organização do Sistema

O sistema é organizado em módulos independentes implementados em Go. Os principais componentes são:

- **Broker:** Gerencia o estado global, sincroniza eventos via Lamport, executa a herança em anel (Ring Failover) de setores órfãos e coordena o despacho de drones.
- **Sensor:** Simula dispositivos ambientais que detectam ocorrências e as propagam via Multicast.
- **Drone:** Agente autônomo de resposta com lógica de *fallback* e modelo de falha probabilístico.
- **Monitor:** Dashboard tático que consolida a visão da malha via WebSockets com lógica de de-duplicação de eventos.

3.3. Protocolos de Rede e Justificativas

A escolha dos protocolos seguiu critérios de resiliência e eficiência para cada perfil de tráfego:

- **UDP Multicast (Porta 9876, endereço 224.1.2.3):** Utilizado para telemetria de sensores. A justificativa reside na necessidade de redundância: ao publicar em um grupo *multicast*, o sensor garante que todos os *brokers* ativos capturem o alerta sem precisar conhecer o IP de cada um.
- **TCP P2P e Drone-Broker (Portas 6000-6008):** Os *brokers* mantêm conexões TCP diretas entre si para a propagação de relógios de Lamport, heartbeats e mensagens de coordenação. A mesma porta é compartilhada para o registro e controle de drones, otimizando o uso de sockets do sistema. O TCP garante a entrega confiável de mensagens de sincronização.

3.4. Implementação da Concorrência

A concorrência é o cerne do *broker*, que precisa lidar simultaneamente com a rede P2P, os sensores e os drones. O uso de *goroutines* permite que cada conexão seja tratada em um fio de execução independente. Para garantir a segurança de memória ao acessar a fila de ocorrências compartilhada, utilizou-se *mutexes*. Abaixo, um trecho de código ilustra o processamento concorrente de mensagens Lamport:

```
func (b *Broker) handleP2PConnection(conn net.Conn) {
    defer conn.Close()
    decoder := json.NewDecoder(conn)
    for {
        var msg P2PMessage
        if err := decoder.Decode(&msg); err != nil { break }

        b.mu.Lock()
        // Sincronização do Relógio de Lamport
        if msg.Timestamp > b.lamportClock {
            b.lamportClock = msg.Timestamp
        }
        b.lamportClock++
        b.mu.Unlock()

        b.processEvent(msg)
    }
}
```

3.5. Decisões de Design e Resiliência

Uma decisão crítica foi o modelo de *fallback* dos drones. Caso um *broker* falhe, o drone tenta reconexão com o próximo *broker* da lista configurada. Além disso, implementou-se a exclusão mútua distribuída baseada na ordem de Lamport: todos os *brokers* enxergam a mesma fila, mas apenas o *broker* que possui a conexão ativa com o drone mais próximo executa o despacho, notificando os demais para removerem a ocorrência da fila.

Outra decisão importante foi a inclusão de um modelo de falha realístico: drones possuem 10% de chance de serem "abatidos" durante uma missão. Esta lógica força o sistema a tratar a perda de agentes de atuação e reatribuir missões, testando a robustez da malha sob condições adversas.

Para garantir a estabilidade do sistema sob o modelo de falhas com alta taxa de recomposição (onde drones redefinem conexões rapidamente após abates), foi implementada uma proteção contra condições de corrida nas conexões TCP persistentes. Nos blocos de encerramento (*defer*) dos loops de leitura de drones e de brokers vizinhos, a remoção do nó correspondente dos mapas ativos do sistema é realizada apenas se a conexão que está sendo fechada corresponder exatamente à conexão registrada no mapa. Caso um nó se reconecte sob uma nova conexão antes do encerramento da goroutine da conexão anterior, a remoção e marcação de indisponibilidade é ignorada para a conexão antiga, impedindo que o nó ativo seja indevidamente desvinculado ou marcado como offline.

4. Resultados e Discussões

4.1. Ambiente de Testes

A validação do HormuzNet foi conduzida utilizando o *Docker Compose* para simular um ambiente distribuído em rede local. Foram instanciados 9 *brokers*, 7 drones autônomos e 18 sensores (2 para cada setor). O monitoramento foi realizado através do dashboard tático acessível via navegador na porta 8085.

4.2. Análise de Resiliência e Consistência

A Tabela 1 apresenta os resultados dos testes de estresse e falhas injetadas no sistema para verificar a capacidade de recuperação da malha.

Table 1. Testes de resiliência e comportamento da malha.			
ID	Cenário de Falha	Resposta do Sistema	Status
TR-01	Queda do Broker B3 (Nordeste)	B2 assume Setor Nordeste via Ring Failover; drones fazem <i>fallback</i>	OK
TR-02	Sensor Multicast	Alerta recebido simultaneamente por todos os brokers	OK
TR-03	Drone Abatido	Broker detecta perda e reenvia alerta para a fila global	OK
TR-04	Conflito Lamport	Brokers ordenam ocorrências identicamente via timestamps	OK
TR-05	Partição de Rede	Brokers isolados operam regionalmente e sincronizam ao retornar	OK

4.3. Desempenho da Sincronização

As métricas de latência e processamento foram coletadas durante uma simulação de 10 minutos com alta taxa de ocorrências (1 evento a cada 2 segundos). Os resultados estão resumidos na Tabela 2.

Table 2. Métricas de desempenho e sincronização.

Métrica	Valor Observado	Observação
Tempo de Despacho (médio)	≈ 1.5 s	Da ocorrência ao envio do drone
Divergência de Lamport	0 ms	Filas mantiveram-se idênticas
Uso de CPU por Broker	$< 5\%$	Eficiência das goroutines
Taxa de Sucesso de Missões	$\approx 91\%$	Impacto do modelo de 10% de falha
Latência P2P entre Brokers	< 10 ms	Comunicação TCP em rede local

4.4. Discussão dos Resultados

Os experimentos demonstraram que a descentralização atingiu o objetivo de eliminar SPOFs. A queda de um *broker* (ex: B3) não interrompeu o monitoramento de sua região, pois o vizinho imediato (B2) assumiu a herança do setor órfão via Ring Failover em tempo real, enquanto o *fallback* dos drones para nós vizinhos ocorreu sem perdas. A consistência garantida pelo algoritmo de Lamport e pela exclusão mútua distribuída (ownership da conexão TCP de cada drone) evitou que múltiplos drones atendessem a mesma ocorrência. Adicionalmente, a de-duplicação de eventos implementada no Monitor permitiu manter a interface táctica limpa e livre de duplicatas de fofoca, mesmo sob alta carga de ocorrências concorrentes.

Além disso, a implementação de validação de identidade da conexão ativa nos blocos de limpeza resolveu o problema de esgotamento de drones disponíveis que surgia a longo prazo. Anteriormente, com a alta rotatividade gerada por abates, conexões obsoletas que se encerravam tardiamente desativavam a chave dos drones nos brokers locais, deixando-os inativos de forma permanente mesmo marcados como disponíveis. Com a correção das condições de corrida de limpeza de conexão, a malha demonstrou capacidade de rodar indefinidamente mantendo o despacho contínuo de tarefas sem acúmulo de fila pendente.

A principal limitação observada foi a escalabilidade do *multicast* em redes que não suportam IGMP de forma eficiente, o que poderia restringir o sistema em ambientes de rede WAN complexos. Contudo, no escopo de uma rede de monitoramento regional, a solução mostrou-se extremamente robusta.

5. Conclusão

O desenvolvimento do HormuzNet permitiu a consolidação de conceitos fundamentais de sistemas distribuídos em uma aplicação prática e crítica. A transição de uma arquitetura centralizada para uma malha descentralizada provou ser eficaz para aumentar a resiliência do monitoramento marítimo no Estreito de Ormuz.

As decisões de design, como o uso de UDP Multicast para aquisição redundante e Relógios de Lamport para consistência de estado, foram validadas pelos testes de estresse e injeção de falhas. O sistema demonstrou capacidade de auto-recuperação e manutenção

da integridade operacional mesmo diante da perda de componentes vitais, como *brokers* e drones.

Trabalhos futuros poderiam explorar a integração de algoritmos de eleição de líder para coordenação de tarefas mais complexas e o uso de criptografia na malha P2P para garantir a segurança da comunicação tática. Em suma, o HormuzNet estabelece uma base sólida para sistemas de monitoramento autônomos em ambientes de alta criticidade.

References

- Deering, S. (1989). Host extensions for ip multicasting. RFC 1112, RFC Editor.
- Donovan, A. A. and Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley Professional.
- Institute, I. S. (1981). Transmission control protocol. RFC 793, RFC Editor.
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565.
- Tanenbaum, A. S., Wetherall, D. J., and Feamster, N. (2021). *Redes de Computadores*. Pearson.